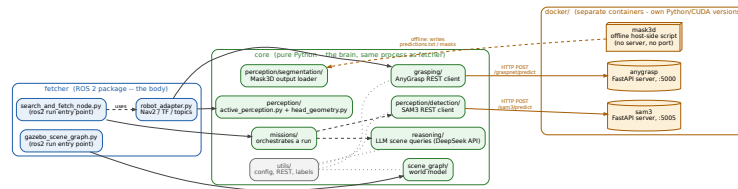


# OVMM: System Walkthrough and Approaches

## The system in one paragraph

A fetch command goes through one loop, repeated until the object is grasped or every candidate location is exhausted: *reason* about where the object probably is (scene graph lookup, or an LLM guess if it isn't known) → *navigate* to a safe standoff near that location → *perceive* (look, detect with SAM3, re-aim if needed) → *grasp* (AnyGrasp pose → arm joints). **core/** is plain Python with no ROS dependency (the reasoning/planning side); **fetcher** is the ROS 2 package that turns **core/**'s decisions into real Nav2 goals, head trajectories, and arm motions.



## What each folder does

**perception/** SAM3 detection client, Mask3D output loader, and the pure geometry for where to look/stand (**active\_perception.py**, **head\_geometry.py**).

**scene\_graph/** The world model: furniture/objects as nodes (centroid, dimensions), connected by “sits on” edges. Built once per scene.

**grasping/** AnyGrasp REST client - point cloud in, ranked collision-aware grasp poses out.

**reasoning/** Talks to the LLM: known-location lookup or top-*k* furniture prediction when the object isn't in the graph yet.

**missions/** Orchestrates one fetch attempt end to end against a **RobotAdapter** protocol it never implements itself.

**utils/** Config loading, the REST client base class, label tables.

**fetcher/** The ROS 2 side: **robot\_adapter.py** is the only place Nav2/TF/topics/joint trajectories appear (it implements **RobotAdapter** for the HSR); the other two files are thin **ros2 run** entry points calling straight into **core/**.

## Approaches

Each of these replaced a version that looked reasonable but broke on real geometry - the common thread is trusting live data (TF, the costmap, a fresh detection) over a single fixed number computed

once and assumed to still be true.

**World↔map frames.** The scene graph stores `world`-frame ground truth; Nav2 plans in `map`, anchored at the robot’s fixed spawn point  $t = (5.0, 6.6, 0)$ . Every pose is converted via  $p_{\text{map}} = p_{\text{world}} - t$  through a TF lookup, not assumed to already coincide.

**Reachability-aware standoff.** Instead of a fixed offset from a target, `reachable_approach_pose` scans an annulus of candidate points  $(c_x + r \cos \theta_i, c_y + r \sin \theta_i)$  for  $r \in [r_{\min}, r_{\max}]$  (closest-first) against the live costmap, accepting the first point with cost  $< 253$  (`nav2_costmap_2d`’s inscribed-obstacle threshold).  $r_{\min}/r_{\max}$  are not fixed either: `_reach_band()` computes them as a clearance constant (0.3/0.55 m,  $\approx$  the robot’s footprint radius) *plus* the target furniture’s own `approach_radius` - half its *shorter* horizontal dimension, since an approach is normally perpendicular to a piece’s long axis. A small table and a long counter each get a standoff scaled to their own size.

**Height-aware look-at target.** A resting object’s aim point used its furniture’s centroid  $z$  - the volumetric middle of the whole piece - not where objects actually sit. For furniture with one flat top (table/desk/coffee table/trolley) the target is corrected to  $z_{\text{top}} = c_z + h/2$ . Multi-level storage (shelf/cabinet) keeps the bulk centroid instead, since an object can be on any internal level and the bounding-box top would aim above all of them.

**Active re-centering.** The first `look_at` aims at the a-priori estimate, not the object itself, so if navigation landed a bit off, that first detection can be off-center. After it succeeds, its mask is lifted to a point cloud, the centroid is converted back to world frame, and `look_at` fires once more at the object’s *actual* position before the final detection is used - re-sensing instead of trusting the original estimate, with the original detection kept as a fallback if re-detection fails.

**Nearest-of-duplicates disambiguation.** When a query matches a label that isn’t unique in the scene (two pringles cans), the LLM has no spatial awareness to choose between them. The match is now overridden with  $\arg \min_i \|c_i - p_{\text{robot}}\|_2$  over every item sharing that label, using the robot’s live position - a geometric tie-break instead of an arbitrary language-model one.

**Two-phase grasp motion.** Arm joints used to move together in one trajectory, so the gripper could sweep forward while still rising - risking a clip across whatever surface the object rests on. The arm now lifts to the target height first with `arm_flex_joint` held at its retracted limit (0.0), *then* extends to the full target pose.

**Known limitation.** The arm-joint formula only ever uses the grasp’s *position*; AnyGrasp’s chosen *orientation* is discarded; and `wrist_flex/arm_roll` are fixed constants. This is the next real fix needed, not yet attempted live - the formula is an analytic approximation, not inverse kinematics.